



TASK SPEC · BACKEND

SecureChat

Plataforma de Chat com Criptografia Ponta-a-Ponta (E2EE)

Requisitos funcionais · Fluxos · Modelo de classes · Estrutura de projeto

AUTOR / PO

Bernardo

STACK

Java 17 · Spring Boot 3 · Kafka · Docker

ENTREGA

MVP em sprints incrementais

VERSÃO

1.0

1. Contexto e Objetivo

O **SecureChat** é uma plataforma de mensagens instantâneas com **criptografia ponta-a-ponta (E2EE)**. O servidor atua apenas como intermediário de roteamento e persistência de dados cifrados — em nenhum momento ele tem acesso ao conteúdo em texto claro das mensagens nem às chaves privadas dos usuários.

*Princípio central: **criptografia híbrida**. Cada mensagem é cifrada com uma chave simétrica (AES-256-GCM) gerada por mensagem; essa chave simétrica, por sua vez, é cifrada com a chave pública do destinatário (RSA-2048 ou ECDH P-256). Apenas quem possui a chave privada correspondente consegue decifrar a chave AES e, consequentemente, a mensagem.*

Objetivo da task: implementar o backend (Spring Boot) responsável pelo cadastro de usuários e chaves públicas, roteamento de mensagens cifradas, entrega em tempo real via WebSocket e persistência auditável — sem jamais decifrar conteúdo.

2. Requisitos Funcionais

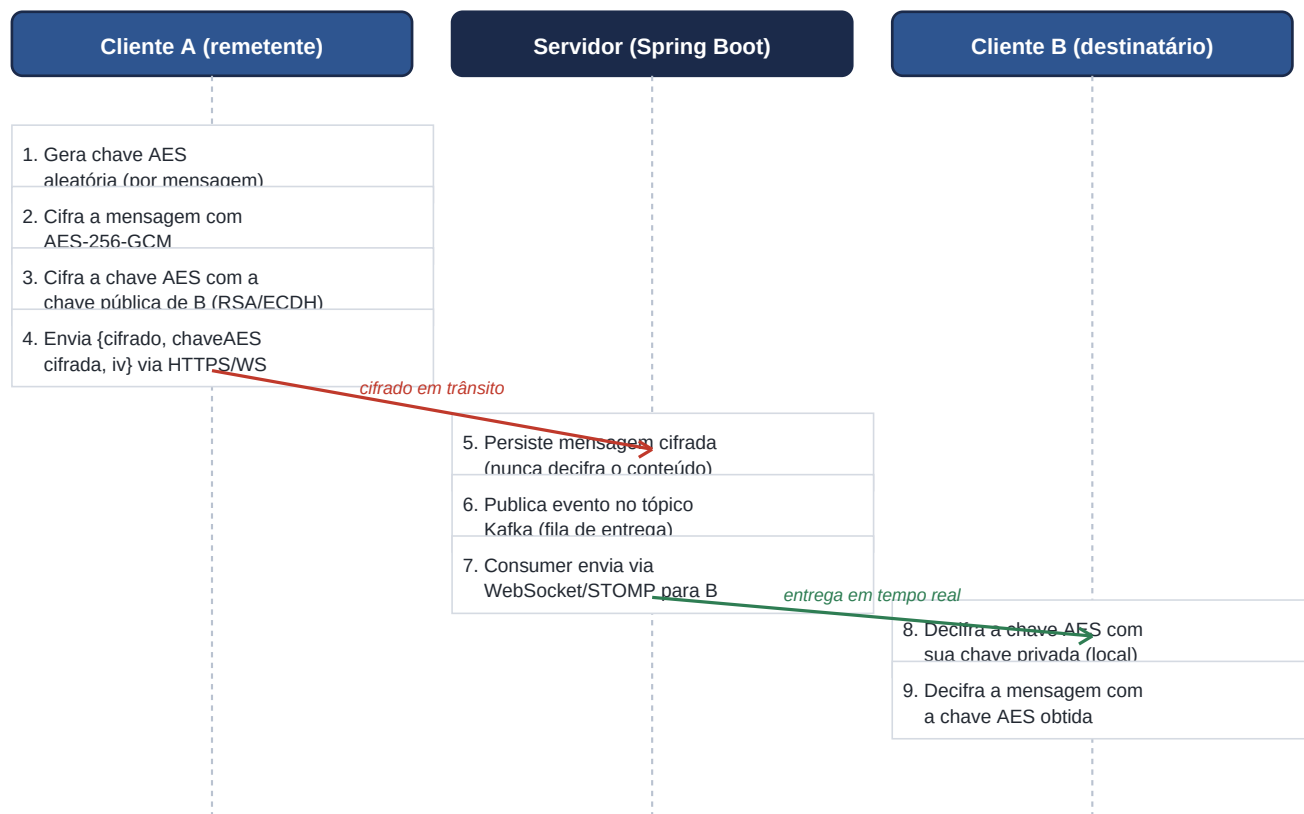
ID	Descrição	Prioridade
RF01	Registrar usuário e associar sua chave pública (gerada no cliente)	Alta
RF02	Consultar chave pública de um usuário por username/e-mail	Alta
RF03	Enviar mensagem cifrada: {conteúdo cifrado, chave AES cifrada, IV}	Alta
RF04	Receber mensagens em tempo real via WebSocket (STOMP)	Alta
RF05	Persistir mensagens cifradas em banco relacional (auditável)	Alta
RF06	Listar histórico de conversa com paginação	Média
RF07	Indicar presença online/offline do usuário	Média
RF08	Confirmação de entrega e leitura (status da mensagem)	Média
RF09	Suporte a múltiplos dispositivos por usuário (sessões)	Baixa
RF10	Rotação/renovação de chaves públicas (re-key)	Baixa

Requisitos Não-Funcionais

ID	Descrição
RNF01	O servidor nunca armazena nem tem acesso a chaves privadas
RNF02	Toda comunicação cliente-servidor ocorre sobre TLS
RNF03	Criptografia simétrica: AES-256-GCM (autenticada)
RNF04	Criptografia assimétrica: RSA-2048 ou ECDH sobre curva P-256
RNF05	Ambiente local 100% reproduzível via Docker Compose
RNF06	Rate limiting no endpoint de envio de mensagens (anti-flood)
RNF07	Logs de aplicação nunca registram conteúdo cifrado nem chaves

3. Fluxo de Funcionamento

O diagrama abaixo resume o ciclo de vida de uma mensagem, do remetente ao destinatário, destacando em quais pontos a cifragem/decifragem acontece — sempre no cliente, nunca no servidor.



O servidor nunca tem acesso a chaves privadas nem ao conteúdo em texto claro.

Fluxo de Registro e Troca de Chaves

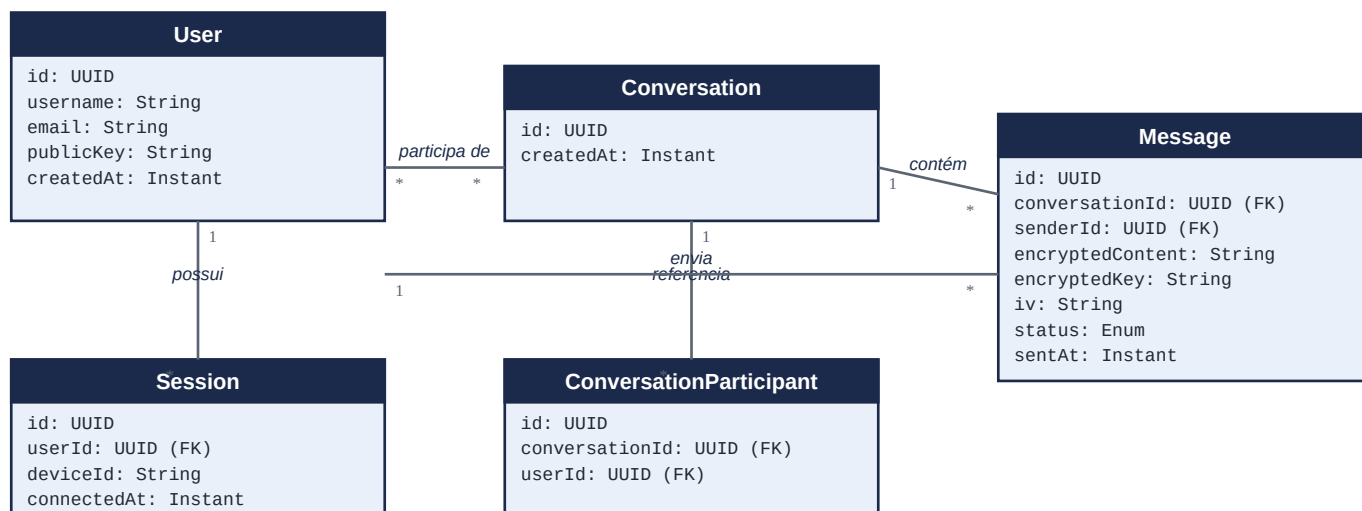
- Cliente gera par de chaves (pública/privada) localmente ao criar a conta.
- Cliente envia ao servidor: username, e-mail e apenas a chave pública.
- Servidor persiste a chave pública associada ao usuário; a privada nunca sai do dispositivo.
- Ao iniciar uma conversa, o cliente busca a chave pública do destinatário via API antes de cifrar a primeira mensagem.

Fluxo de Conexão em Tempo Real

- Cliente autentica via JWT (obtido no login) e abre handshake WebSocket/STOMP.
- Servidor valida o token, registra a sessão (dispositivo + usuário) e marca presença como online.
- Mensagens novas são publicadas em um tópico Kafka; um consumer entrega via WebSocket a quem estiver conectado.
- Se o destinatário estiver offline, a mensagem cifrada fica persistida e é entregue no próximo login.

4. Modelo de Classes e Relacionamentos

Modelo de domínio simplificado para o MVP. Todas as entidades relacionadas a mensagens armazenam apenas dados cifrados — o campo `encryptedContent` nunca contém texto plano.



Descrição das Entidades

Entidade	Responsabilidade
User	Representa um usuário cadastrado. Guarda a chave pública, nunca a privada.
Session	Sessão ativa de um dispositivo específico (suporte multi-device e presença).
Conversation	Agrupa mensagens trocadas entre dois ou mais participantes.
ConversationParticipant	Tabela associativa N:N entre User e Conversation.
Message	Mensagem cifrada: conteúdo, chave AES cifrada, IV e status de entrega.

Relacionamentos

- **User 1—N Session**: um usuário pode ter várias sessões ativas (múltiplos dispositivos).
- **User N—N Conversation** (via **ConversationParticipant**): um usuário participa de várias conversas e vice-versa.
- **Conversation 1—N Message**: cada conversa contém várias mensagens.
- **User 1—N Message** (como remetente): cada mensagem tem exatamente um autor.

5. Estrutura do Projeto (Backend — Spring Boot)

Organização em camadas, alinhada ao que já é usado no SchemaGuard, facilitando reaproveitamento de padrões e configuração.

```
com.bernardo.securechat
├── config
│   ├── WebSocketConfig.java # STOMP endpoints, broker
│   ├── SecurityConfig.java # JWT, CORS, rotas públicas
│   └── KafkaConfig.java # tópicos, producers/consumers
├── controller
│   ├── AuthController.java # /auth/register, /auth/login
│   ├── UserController.java # /users/{id}/public-key
│   └── MessageController.java # /conversations/{id}/messages
├── domain
│   ├── User.java
│   ├── Session.java
│   ├── Conversation.java
│   ├── ConversationParticipant.java
│   └── Message.java
├── repository
│   ├── UserRepository.java
│   ├── ConversationRepository.java
│   └── MessageRepository.java
├── service
│   ├── UserService.java
│   ├── ConversationService.java
│   ├── MessageService.java # apenas roteia/persiste, nunca decifra
│   └── PresenceService.java
├── websocket
│   └── ChatStompController.java # @MessageMapping /chat.send
├── kafka
│   ├── MessageProducer.java
│   └── MessageConsumer.java
├── security
│   ├── JwtUtil.java
│   └── JwtAuthFilter.java
├── dto
│   ├── SendMessageRequest.java # encryptedContent, encryptedKey, iv
│   └── PublicKeyResponse.java
├── exception
└── GlobalExceptionHandler.java
```

Docker Compose (ambiente local)

```
services:
  api: # Spring Boot app (porta 8080)
  postgres: # mensagens cifradas + chaves públicas
  redis: # sessões WebSocket / presença online
  kafka: # fila de entrega assíncrona
  kafka-ui: # (opcional) inspeção de tópicos em dev
```

6. Como o Sistema Funciona, de Ponta a Ponta

Resumo executivo do comportamento completo do sistema, para dar contexto de produto além da implementação técnica:

- **Cadastro:** o app cliente gera o par de chaves no primeiro acesso e envia apenas a chave pública ao servidor.
- **Início de conversa:** ao selecionar um contato, o cliente busca a chave pública dele via API.
- **Envio:** o cliente cifra a mensagem localmente (AES) e cifra a chave AES com a chave pública do destinatário (RSA/ECDH); o servidor recebe apenas dados cifrados.
- **Roteamento:** o servidor persiste a mensagem e publica um evento no Kafka; um consumer entrega via WebSocket para quem estiver online.
- **Recebimento:** o cliente destinatário decifra a chave AES com sua chave privada (local) e, em seguida, decifra a mensagem.
- **Offline:** mensagens para usuários offline ficam persistidas cifradas e são entregues (ou buscadas via histórico) no próximo login.

Nota de escopo: este é um projeto educacional/portfólio. Para produção real, seria necessário endereçar forward secrecy, autenticação de chave pública contra ataques man-in-the-middle (ex.: verificação de fingerprint, como no Signal Protocol) e gerenciamento de prekeys — itens fora do escopo deste MVP, mas bons pontos de discussão em entrevista técnica.

Definition of Done (MVP)

- RF01 a RF05 implementados e testados (unitário + integração com Testcontainers).
- Nenhum texto plano ou chave privada trafega ou é logado no backend.
- docker-compose up sobe o ambiente completo sem configuração manual adicional.
- README documenta o modelo de ameaça e as limitações conhecidas do MVP.